

PROJECT INTERNSHIP REPORT

On

CREATOR COLLABORATION HUB

A Collaborative Platform for Developers, Designers, and Editors

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

SOFTWARE ENGINEERING

Submitted By:**Under the Guidance of:**

Student Name

Guide Name

Enrollment / Roll No.

Designation

Semester / Year

Department of Computer Science

Department of Computer Science and Engineering

[University/College Name]

[Year]

Table of Contents

3. Task Performed	3
3.1 Introduction to Technology	3
3.2 Feasibility Study	5
3.3 Tools and Technologies Used	6
3.4 Modules	8
3.5 Task Performed	10
3.6 Objectives of the Project	11
3.7 System Requirement	12
3.8 Functional Requirements	12
3.9 Non-Functional Requirements	13
3.10 Design and Implementation of the System	14
3.11 Screenshots	16
3.12 Testing	18
4. Reflection (Specific Outcomes)	20
4.1 Technical Outcomes of the Project	20
4.2 Non-Technical Outcomes of Projects	21
5. Conclusion	22
6. Future Scope	23
7. References	24

3. Task Performed

This section outlines the detailed tasks performed during the development of the Creator Collaboration Hub. The Creator Collaboration Hub is a comprehensive web application engineered to bridge the gap between creative professionals, including software developers, UI/UX designers, and video editors. By providing a unified platform, it streamlines the process of discovering projects, applying for roles, managing team collaboration, and executing tasks efficiently.

3.1 Introduction to Technology

The development of modern web applications requires a robust stack capable of handling high-performance interactions, secure data transmission, and scalable architectures. The Creator Collaboration Hub was built using a state-of-the-art tech stack comprising React.js for the frontend, Spring Boot for the backend, and MySQL for persistent data storage.

3.1.1 Frontend: React.js and Tailwind CSS

React.js is an open-source, component-based JavaScript library maintained by Facebook, utilized for building highly interactive user interfaces. It utilizes a Virtual DOM (Document Object Model) to optimize rendering performance by only updating components that experience state changes. For this project, React was combined with Vite, a modern frontend build tool that provides lightning-fast hot module replacement (HMR) and optimized build processes.

Tailwind CSS was adopted as the primary styling framework. Unlike traditional CSS frameworks that provide pre-designed components, Tailwind provides low-level utility classes that allow for rapid UI development without leaving the HTML/JSEX structure. This enabled the creation of a bespoke "glassmorphism" aesthetic characterized by translucent backgrounds, blurred backdrops, and modern typography, achieving a premium, startup-grade user experience.

3.1.2 Backend: Spring Boot

Spring Boot is an extension of the Spring framework that simplifies the setup and development of Java-based enterprise applications. It adheres to the "convention over

configuration" paradigm, eliminating boilerplate code and XML configurations. Spring Boot was selected due to its robust ecosystem, built-in security features via Spring Security, and seamless integration with relational databases using Spring Data JPA. The backend exposes a RESTful API architecture that communicates asynchronously with the frontend via JSON payloads.

3.1.3 Database: MySQL

MySQL, a widely-used relational database management system (RDBMS), serves as the central data repository. It ensures ACID (Atomicity, Consistency, Isolation, Durability) compliance, which is critical for maintaining the integrity of user profiles, project applications, and workspace messaging. Spring Data JPA acts as an Object-Relational Mapping (ORM) layer, translating Java entity objects into MySQL database tables dynamically.

3.1.4 Security: JSON Web Tokens (JWT)

Authentication and authorization are managed using JSON Web Tokens (JWT). JWT provides a stateless security mechanism where the server does not need to store session data. Upon successful login, the backend issues a signed JWT containing the user's claims and roles. The frontend stores this token and attaches it as a Bearer token in the Authorization header of subsequent API requests, ensuring secure route protection and identity verification.

3.2 Feasibility Study

Before initiating the development phase, a comprehensive feasibility study was conducted to evaluate the viability of the Creator Collaboration Hub across technical, operational, and economic dimensions. This study ensured that the proposed system could be successfully implemented within the allocated constraints.

3.2.1 Technical Feasibility

Technical feasibility assesses whether the required technology, hardware, and software are available to support the system. The project leverages open-source technologies (React, Java, Spring Boot, MySQL) which are freely accessible and boast extensive community documentation. The development machines met the prerequisite specifications for running local servers and database instances. Furthermore, the modular architecture ensures that future technological integrations, such as real-time WebSockets or third-party APIs (e.g., Cloudinary for media storage), can be accommodated without architectural overhauls. Therefore, the project was deemed highly technically feasible.

3.2.2 Operational Feasibility

Operational feasibility evaluates how well the proposed system solves the identified problem and whether it will be adopted by the target user base. Creative professionals currently suffer from fragmented workflows—using separate applications for finding work (e.g., Upwork), communicating (e.g., Slack/Discord), and managing tasks (e.g., Trello). By consolidating project discovery, application management, and workspace collaboration into a single, cohesive platform, the system dramatically reduces operational friction. The intuitive UI/UX design minimizes the learning curve, ensuring high operational feasibility and user adoption rates.

3.2.3 Economic Feasibility

Economic feasibility involves a cost-benefit analysis. As an academic project/internship prototype, the primary investment was intellectual capital and time. The use of open-source frameworks and free-tier cloud deployment solutions (such as GitHub for version control and Cloudinary for image hosting) eliminated licensing and software costs. The long-term economic benefits of such a platform include potential monetization through premium features, highlighting the project's viability from an economic standpoint.

3.3 Tools and Technologies Used

The successful execution of the Creator Collaboration Hub relied on a curated selection of industry-standard tools and technologies. These tools facilitated efficient coding, robust testing, version control, and seamless deployment workflows.

Category	Tool / Technology	Purpose / Usage in Project
Frontend Framework	React.js (v18) & Vite	Building the Single Page Application (SPA), state management, and component architecture.
Styling & UI	Tailwind CSS, Framer Motion, Lucide Icons	Implementing responsive glassmorphism designs, micro-animations, and modern vector iconography.
Backend Framework	Java 17, Spring Boot (v3.2)	Handling business logic, REST API endpoints, and server-side processing.
Database & ORM	MySQL, Spring Data JPA, Hibernate	Relational data storage, automated schema generation, and entity management.
Security	Spring Security, JWT (jjwt)	Stateless authentication, password encoding (BCrypt), and role-based route protection.
Media Storage	Cloudinary API	Securely hosting user avatars, project banners, and portfolio media assets.
Version Control	Git, GitHub	Source code management, branching strategies, and tracking project history.
Development Tools	VS Code, IntelliJ IDEA, Maven, npm	Integrated Development Environments (IDEs) and package management.

Category	Tool / Technology	Purpose / Usage in Project
API Testing	Postman / Swagger UI	Testing and documenting backend REST endpoints during development.

The synergy between these tools established a modern development pipeline. The decoupling of the React frontend from the Spring Boot backend allowed for independent scaling and testing. Maven managed the Java dependencies and build lifecycle efficiently, while NPM handled the JavaScript ecosystem packages required for the Vite React application.

3.4 Modules

To ensure maintainability and separation of concerns, the Creator Collaboration Hub was divided into distinct, loosely coupled modules. Each module encapsulates specific business logic and user workflows.

3.4.1 Authentication & Authorization Module

This module manages user registration, login, and identity verification. It handles the secure hashing of user passwords using BCrypt before storing them in the MySQL database. Upon successful login, it generates a JWT containing the user's ID and role. It also manages authentication middleware (filters in Spring Boot, higher-order components in React) to protect private routes, ensuring that unauthenticated users are redirected to the login interface.

3.4.2 User Profile & Portfolio Module

This module allows creators to establish their professional identity. Users can upload avatars, define their biographical information, and specify their technical skills. The portfolio sub-module enables users to upload media showcasing their past work (images, code snippets, video links), effectively serving as a dynamic resume. This data is critical for project owners evaluating potential collaborators.

3.4.3 Project Management Module

The core feature for project owners. It facilitates the creation, updating, and deletion of collaboration projects. Owners can define project titles, descriptions, required roles (e.g., "Frontend Developer", "Video Editor"), category tags, and team sizes. The module tracks the lifecycle status of a project from `OPEN` (accepting applications) to `COMPLETED`.

3.4.4 Application & Recruitment Module

This module handles the workflow of matching creators to projects. Applicants can browse an "Explore" feed of open projects and submit applications detailing their specific pitch and requested role. Project owners utilize a "Manage Applicants" dashboard to review portfolios, and update application statuses to `SHORTLISTED`,

`ACCEPTED`, or `REJECTED`. The system automatically generates notifications alerting applicants of status changes.

3.4.5 Collaboration Workspace Module

Once an applicant is accepted, this module activates. It generates an exclusive, secure workspace environment isolated to the accepted team members and the project owner. It features a real-time discussion feed for intra-team communication, an official updates board managed by the owner, and a team directory. This module replaces the need for external communication tools, centralizing the collaboration effort.

3.5 Task Performed

The development of the Creator Collaboration Hub was executed in a structured, phased approach, adhering to Agile software development methodologies. The primary tasks performed included:

- **Requirement Gathering & Architecture Design:** Drafted comprehensive user stories defining the roles of Project Owners and Applicants. Designed the relational database schema, mapping entities such as Users, Projects, Applications, Workspaces, and Notifications using Entity-Relationship (ER) diagrams.
- **Backend API Development:** Initialized the Spring Boot application. Created JPA entity classes with appropriate relational mappings (`@OneToMany`, `@ManyToOne`). Developed RESTful controllers for authentication, project CRUD operations, applicant management, and workspace messaging. Implemented a robust global exception handling mechanism to return structured JSON error responses.
- **Security Implementation:** Configured Spring Security filter chains. Implemented JWT generation upon login and token validation filters for intercepting and authenticating incoming HTTP requests. Handled CORS (Cross-Origin Resource Sharing) configurations to allow communication between the React development server and the backend.
- **Frontend UI/UX Engineering:** Scaffolded the Vite React application. Established a global CSS design system utilizing Tailwind CSS to achieve a dark-themed, glassmorphic aesthetic. Developed highly reusable React components (modals, cards, navigation bars) integrated with Framer Motion for fluid transitions and micro-animations.
- **Integration & State Management:** Connected the frontend application to the backend REST APIs using the Axios HTTP client. Implemented React Context API for global state management regarding the authenticated user's session. Handled asynchronous data fetching and loading states gracefully across all dashboards.
- **Collaboration Workflow Implementation:** Engineered the complex state logic required for the "Close Project" and "Workspace Access" workflows. Implemented fallback mechanisms ensuring that legacy database records maintained synchronized access rights to communication channels.

- **Deployment Preparations:** Conducted extensive testing and debugging to resolve connection proxy errors and MySQL Enum constraints. Structured the codebase for eventual containerization and cloud deployment.

3.6 Objectives of the Project

The primary objectives driving the conceptualization and development of the Creator Collaboration Hub are:

1. **Centralize the Creative Process:** To eliminate the fragmentation experienced by creators who must use separate platforms for networking, job hunting, and team communication, by providing a singular, cohesive ecosystem.
2. **Facilitate Skill-Based Networking:** To create an environment where connections are forged based on demonstrable skills and portfolio quality rather than superficial metrics, promoting meritocratic collaborations.
3. **Provide Professional Management Tools:** To equip project leaders and startup founders with an intuitive suite of tools capable of managing the entire recruitment lifecycle—from posting project requirements to shortlisting candidates and finalizing a team.
4. **Enhance Team Productivity:** To offer dedicated, isolated project workspaces that provide distraction-free communication channels and official project updates, ensuring all team members remain aligned on project goals.
5. **Demonstrate Modern Full-Stack Capabilities:** From an academic and engineering perspective, the objective is to demonstrate proficiency in building secure, scalable, and aesthetically premium full-stack web applications utilizing enterprise-grade frameworks.

3.7 System Requirement

To successfully deploy, run, and interact with the Creator Collaboration Hub, the following hardware and software requirements must be met.

Hardware Requirements:

- **Processor:** Intel Core i5 / AMD Ryzen 5 or equivalent (Minimum for development environment).
- **RAM:** 8 GB Minimum (16 GB recommended for running IDE, backend server, and frontend server simultaneously).

- **Storage:** 256 GB SSD (Minimum 1 GB free space for project files and dependencies).
- **Network:** Broadband internet connection for fetching NPM/Maven dependencies and Cloudinary API interactions.

Software Requirements:

- **Operating System:** Windows 10/11, macOS 10.15+, or modern Linux distributions.
- **Backend Environment:** Java Development Kit (JDK) 17.
- **Frontend Environment:** Node.js (v18.x or higher) and npm/yarn.
- **Database:** MySQL Server 8.0+.
- **Web Browser:** Google Chrome, Mozilla Firefox, or Safari (updated to the latest version for CSS backdrop-filter support).

3.8 Functional Requirements

Functional requirements specify the specific behaviors, operations, and functionalities the system must perform. The system was designed to fulfill the following core functional requirements:

- **User Authentication:** The system must allow users to register with an email and password, log in securely, and maintain their session via JWT tokens.
- **Profile Customization:** Users must be able to upload a profile picture, write a biography, and list their technical skills.
- **Project Creation:** Authenticated users must be able to create projects by providing a title, description, deadline, required roles, and an optional banner image.
- **Project Browsing:** The system must provide a global "Explore" feed where users can search for projects by keywords or filter them by active status.
- **Application Submission:** Users must be able to apply to 'OPEN' projects, specifying the role they are applying for and attaching a personalized message/portfolio link.
- **Applicant Management:** Project owners must be able to view all applications for their projects and explicitly update applicant statuses to 'SHORTLISTED', 'ACCEPTED', or 'REJECTED'.
- **Workspace Access Control:** The system must dynamically restrict access to a project's collaboration workspace to solely the project owner and explicitly 'ACCEPTED' team members.
- **Workspace Communication:** Authorized team members must be able to post messages in a real-time discussion feed within the workspace.
- **Notifications:** The system must automatically generate and deliver alerts to users when their application status changes or when a new workspace message is posted.

3.9 Non-Functional Requirements

Non-functional requirements define the system's operational attributes, focusing on performance, usability, security, and reliability.

- **Security:** All API endpoints (except registration/login) must be strictly protected against unauthorized access. Passwords must be hashed utilizing BCrypt, and SQL Injection attacks must be prevented via Spring Data JPA prepared statements.
- **Usability & UI Aesthetics:** The application must utilize a modern "glassmorphism" aesthetic. The interface must be responsive, adapting seamlessly to desktop, tablet, and mobile screen resolutions without breaking the layout.
- **Performance:** Frontend routing must be instantaneous using React Router. The system should load the initial dashboard within 2 seconds under standard network conditions.
- **Maintainability:** The codebase must follow structural best practices (MVC architectural pattern in the backend, functional components with Hooks in the frontend) to ensure future developers can easily comprehend and extend the system.
- **Data Integrity:** The database schema must enforce referential integrity using foreign keys. The system must prevent duplicate application submissions for the same project.

3.10 Design and Implementation of the System

The system was implemented using a decoupled architecture, physically separating the client-side presentation layer from the server-side business logic and data access layer. This design promotes scalability and allows different teams to work on the frontend and backend simultaneously.

3.10.1 System Architecture

The system follows a standard Three-Tier Architecture:

- **Presentation Tier (Frontend):** A React SPA interacting directly with the user. It handles UI rendering, form validation, and state management. It communicates with the backend exclusively via HTTPS REST API calls.
- **Logic Tier (Backend):** The Spring Boot application acts as the intermediary. It receives HTTP requests, processes business logic through dedicated Service classes, enforces security constraints, and prepares data for the presentation tier.
- **Data Tier (Database):** The MySQL relational database stores persistent data. It is directly accessed only by the backend's Data Access Objects (Spring JPA Repositories).

3.10.2 Database Schema Design

The relational database schema is structured around several highly-interconnected entities. The primary tables and their relationships include:

- **users:** Stores core identity and authentication data.
- **collaboration_projects:** Stores project details. Contains a Many-To-One relationship to the `users` table (indicating the creator).
- **collaboration_requests:** Acts as the join table for the application process, mapping `users` (applicants) to `collaboration\_projects` with an associated application status.
- **project_collaborators:** The finalized team roster. Contains records of `users` who have explicitly been granted access to a specific workspace.
- **project_messages & project_updates:** Stores the communication payload linked via Many-To-One relationships to specific projects and users.

- **notifications:** Stores system alerts directed at specific users.

3.10.3 UI/UX Implementation Details

The UI implementation heavily relied on Tailwind CSS to create the platform's signature look. "Glass panels" were created using specific utility combinations such as ``bg-white/10 backdrop-blur-md border border-white/20``, which achieves a frosted glass effect layered over dynamic background gradients. Framer Motion was injected into the component lifecycle to execute entrance animations (e.g., lists fading in and sliding up sequentially) and exit animations for modals, making the application feel highly responsive and "alive" to user inputs.

3.11 Screenshots

The following figures demonstrate the graphical user interface and functional workflows of the Creator Collaboration Hub.

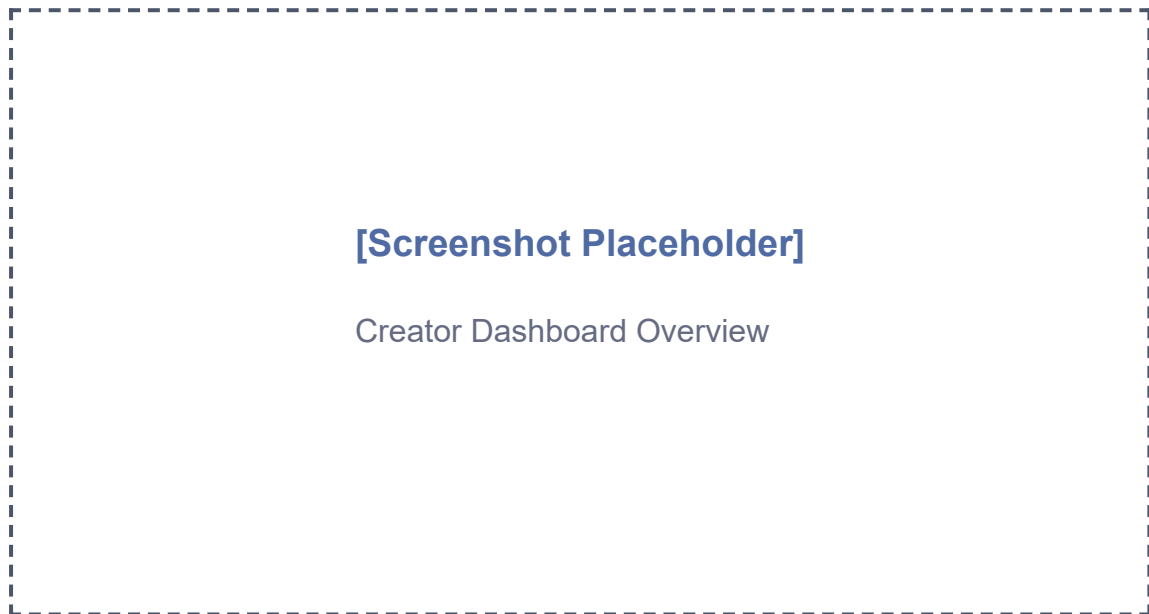


Figure 3.1: The Creator Dashboard, displaying Quick Stats, "My Posted Projects", and Dynamic Tab Navigation.

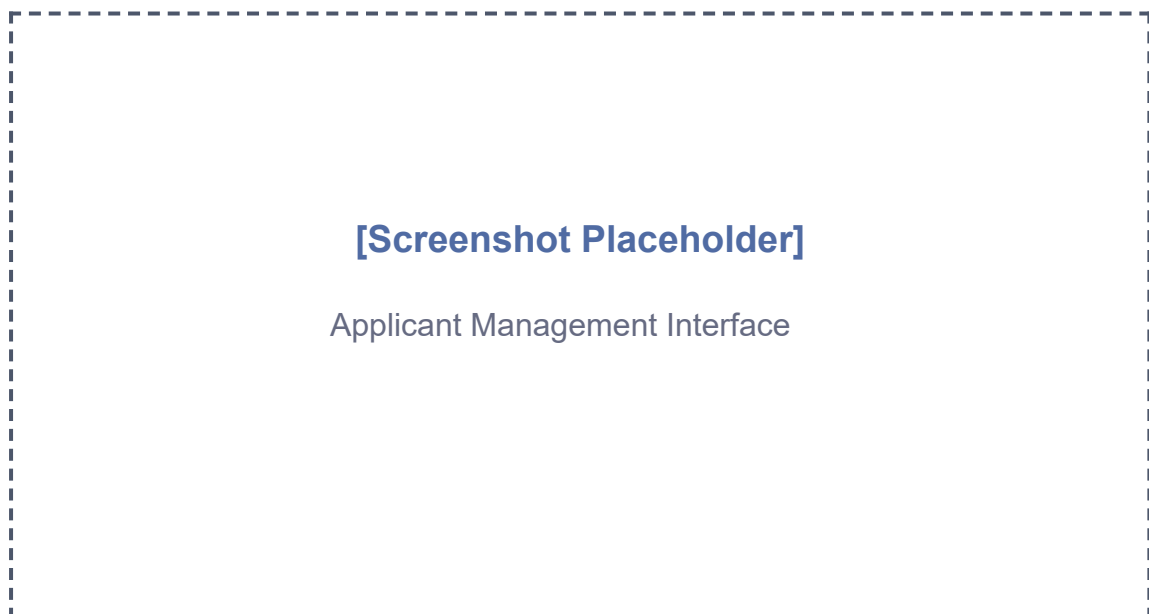


Figure 3.2: The Applicant Management Modal, allowing project owners to review portfolios and Accept/Reject candidates.



Figure 3.3: The isolated Collaboration Workspace featuring real-time messaging, project updates, and team directory.

The interface designs prioritize clarity and visual hierarchy. The use of conditional rendering ensures that users only see action buttons (such as "Enter Workspace") when their backend authorization status strictly permits it, combining UI cleanliness with security best practices.

3.12 Testing

Comprehensive testing methodologies were applied throughout the development lifecycle to ensure the stability, security, and usability of the application. The testing phase aimed to identify and resolve defects before finalizing the platform.

3.12.1 Unit Testing

Unit testing involves isolating individual components or functions to verify their correctness in isolation. In the backend, core service layer methods, such as the `'ProjectService.applyToProject'` logic, were tested to ensure that business rules (e.g., preventing a user from applying to their own project or submitting duplicate applications) threw the correct custom exceptions (`'IllegalArgumentException'` and `'DuplicateResourceException'`). The frontend components were tested for state modifications, ensuring that modal visibility toggles and form input bindings behaved as expected without triggering infinite rendering loops.

3.12.2 Integration Testing

Integration testing evaluates the communication between distinct modules and layers of the architecture. Extensive integration tests were conducted via Postman to verify the REST API endpoints. This included verifying that the Spring Security JWT filters correctly intercepted requests, validated the bearer token, and extracted the `'UserDetails'` object before allowing the request to reach the Controller layer. Additionally, database integration was monitored to ensure that JPA cascading operations correctly persisted related entities (e.g., saving a user automatically created the associated internal tables) and that database constraints, such as MySQL ENUM modifications, were handled smoothly via `ApplicationRunner` migration scripts.

3.12.3 Usability and UI/UX Testing

Usability testing focuses on the end-user experience. The application was subjected to manual navigation tests mimicking actual user workflows. Test scenarios included:

- **The Creator Flow:** Registering an account, creating a project, reviewing incoming applications, accepting an applicant, and navigating to the newly generated workspace.

- **The Applicant Flow:** Browsing the explore page, submitting a detailed application, receiving a notification upon acceptance, and engaging in the workspace discussion.

UI responsiveness was tested across various viewport sizes using browser developer tools to ensure the Tailwind CSS grid and flexbox layouts successfully reorganized content for mobile readability without overflowing containers.

3.12.4 Error Handling Verification

A critical testing phase involved intentionally triggering errors (e.g., submitting malformed JSON, accessing unauthorized resources) to verify the robustness of the `GlobalExceptionHandler`. This ensured that the frontend received predictable, structured JSON error messages rather than raw Java stack traces, allowing the React application to display user-friendly alert popups.

4. Reflection (Specific Outcomes)

The development of the Creator Collaboration Hub provided extensive learning opportunities, bridging theoretical software engineering concepts with practical, production-level implementation. The project yielded significant outcomes across both technical and non-technical domains.

4.1 Technical Outcomes of the Project

The primary technical outcome was mastering the end-to-end integration of a modern tech stack. Specific technical achievements include:

- **Proficiency in RESTful Architecture:** Gained a deep understanding of designing stateless APIs, managing HTTP status codes appropriately, and structuring JSON payloads for optimal client-server communication.
- **Advanced Spring Boot & JPA Mastery:** Successfully implemented complex relational database mappings. Mastered the nuances of Hibernate lazy vs. eager fetching, solving `'LazyInitializationException'` errors within transactional boundaries, and executing custom queries using Spring Data JPA method naming conventions.
- **Modern React Implementation:** Solidified knowledge of functional React components and React Hooks (`'useState'`, `'useEffect'`, Context API). Learned to decouple UI logic from business logic, creating highly reusable component hierarchies.
- **Advanced CSS and Animation:** Acquired practical experience with Tailwind CSS to build complex, responsive layouts rapidly. Mastered the implementation of the glassmorphism design trend and utilized Framer Motion to map React component lifecycles to CSS animations, significantly enhancing UX.
- **Security and Authentication Protocols:** Successfully implemented JWT-based authentication from scratch, understanding token signing, expiration handling, and stateless security contexts within Spring Security filter chains.
- **Debugging and Error Resolution:** Developed strong troubleshooting skills, successfully diagnosing complex full-stack issues such as MySQL data truncation

errors and CORS policy blockages by analyzing server stack traces and browser network payloads.

4.2 Non-Technical Outcomes of Projects

Beyond coding, the project fostered crucial professional and soft skills essential for a software engineering career:

- **Project Management and Scope Definition:** Learned to break down a massive conceptual idea into manageable, actionable tasks. Prioritized core features (MVP) over edge cases to ensure the project was completed within the developmental timeframe.
- **System Thinking:** Developed the ability to view the application holistically, understanding how a change in the database schema cascades through the Java backend and impacts the frontend React component state.
- **User-Centric Design Philosophy:** Cultivated empathy for the end-user. Prioritized clear feedback mechanisms (loading spinners, descriptive error alerts, success notifications) to ensure users always understand the system's current state.
- **Adaptability and Problem Solving:** Demonstrated resilience when facing technical roadblocks (e.g., overriding rigid framework defaults) by consulting documentation, researching community solutions, and applying lateral thinking to engineer functional workarounds.

5. Conclusion

The Creator Collaboration Hub was successfully conceptualized, designed, and implemented as a robust, full-stack web application. It achieves its primary goal of unifying the fragmented landscape of creative networking by providing a centralized platform for project discovery, applicant tracking, and secure team collaboration.

By leveraging the power of React.js and Tailwind CSS, the platform delivers a visually stunning, highly responsive user interface that meets modern design expectations. Simultaneously, the Spring Boot and MySQL backend ensures that data integrity, business logic, and security are handled with enterprise-grade reliability. The implementation of JWT authentication and role-based access control guarantees that project workspaces remain exclusive and secure.

This project serves as a comprehensive demonstration of modern software engineering principles, encompassing architectural planning, database design, API development, and UI/UX engineering. The successful resolution of complex state management challenges and database schema migrations further validates the technical robustness of the system. Ultimately, the Creator Collaboration Hub stands as a highly viable, scalable prototype capable of revolutionizing how digital creators connect and collaborate globally.

6. Future Scope

While the current iteration of the Creator Collaboration Hub provides a complete and functional core workflow, the platform's modular architecture allows for extensive future enhancements. Identified areas for future scope include:

- **Real-Time WebSockets Integration:** Transitioning the Workspace Discussion Feed from REST-based polling to full-duplex WebSockets (e.g., utilizing Spring WebSockets or Socket.io) to enable instantaneous, real-time chat functionality without client-side refresh requirements.
- **Advanced Task Management (Kanban Boards):** Integrating a robust task management module within the workspace, allowing project owners to assign specific tickets or tasks to collaborators, tracking progress via interactive Kanban boards (similar to Jira or Trello).
- **Integrated Code and Asset Sharing:** Building specialized modules within the workspace for developers to link GitHub repositories and for designers to embed Figma files natively, centralizing asset management.
- **Reputation and Review System:** Implementing a post-collaboration review system where project owners and collaborators can rate each other upon project completion. This data would feed into an algorithmic "Creator Rating" displayed on user profiles, building a trust economy.
- **Mobile Application Deployment:** Utilizing frameworks like React Native to package the frontend application into native iOS and Android applications, increasing accessibility and enabling native push notifications for project updates.
- **AI-Powered Matchmaking:** Integrating Machine Learning algorithms to analyze an applicant's portfolio data and technical skills, automatically suggesting highly compatible projects on their Explore feed.

7. References

The successful development of this project was supported by extensive documentation and community resources. The following references were utilized during the research and implementation phases:

1. **Spring Documentation:** Spring Framework and Spring Boot Official Reference Documentation. Available at: <https://spring.io/projects/spring-boot>
2. **React Documentation:** React Official Documentation (Hooks, Context, Virtual DOM). Available at: <https://react.dev/>
3. **Tailwind CSS:** Tailwind CSS Utility-First Framework Documentation. Available at: <https://tailwindcss.com/docs>
4. **Hibernate ORM:** Hibernate User Guide for Java Persistence API. Available at: <https://hibernate.org/orm/documentation/>
5. **JWT Authentication:** JSON Web Token Introduction and Libraries. Available at: <https://jwt.io/introduction>
6. **Vite Build Tool:** Next Generation Frontend Tooling Documentation. Available at: <https://vitejs.dev/guide/>
7. **Framer Motion:** Production-ready React animation library. Available at: <https://www.framer.com/motion/>
8. **MySQL Reference:** MySQL 8.0 Reference Manual (Schema design and optimization). Available at: <https://dev.mysql.com/doc/refman/8.0/en/>

